

The Accidental Messenger

Every developer has written code for a machine that a human has to read tomorrow.

You know the feeling. You stare at a function you wrote six months ago, surrounded by boilerplate you didn't choose, ceremony the language demands, import paths that spell out an organizational decision nobody made consciously. The logic is in there somewhere, buried. The machine never had trouble finding it. The machine doesn't care about the scaffolding. You wrote all of that for the machine, and then you had to read it yourself.

None of the syntax was for you. It was never for you.

The Accident

FORTRAN was designed in 1957 to let scientists write formulas that a compiler could translate to assembly. The formulas looked vaguely mathematical because the scientists were mathematicians. The compiler constraints leaked through. You can see them in the column limits, the line numbering, the restrictions that were there because the IBM 704 punched-card reader needed them, not because they made programs clearer.

C cleaned that up. C gave us structured control flow, functions, named types. It was a revolution. It was also designed for efficient compilation to PDP-11 assembly. The pointer arithmetic, the undefined behavior, the manual memory model: all of it reflects decisions made for the machine. We called it "close to the metal" as though proximity to hardware were a virtue rather than a constraint.

Java tried to fix C. It added garbage collection, bytecode portability, a type system that prevented certain classes of corruption. But Java brought its own ceremony: public static void main, checked exceptions, the verbosity that became the butt of a decade of jokes. It reflected a different set of constraints, enterprise software in the 1990s, distributed systems, teams of hundreds. The machine was now a JVM. The ceremony served that machine.

TypeScript is where most of us live today. It is genuinely good. The type inference is smart, the tooling is excellent, the ecosystem is vast. But TypeScript is JavaScript with types bolted on, and JavaScript was written in ten days in 1995 to make web pages slightly more interactive. The prototype chain, the coercion rules, the `this` semantics: these are the consequences of decisions made in a context that no longer exists, preserved because billions of lines of code depend on them.

Each generation of languages cleaned up the previous accident. None of them questioned the fundamental premise: a programming language is a messenger from humans to compilers. We write; the compiler reads; the machine runs. We adapted to the messenger.

The Inheritance

Then came the large language models.

GPT-4, Claude, Gemini: these systems are remarkably good at generating TypeScript. They write well-structured functions, handle edge cases, name variables with taste. It feels like they understand the code. In some sense, they do.

But here is the irony: they are not good at TypeScript because TypeScript is a good language for AI to generate. They are good at TypeScript because they trained on hundreds of billions of lines of TypeScript written by humans adapting to compilers. They learned the accident. They internalized the ceremony, the boilerplate patterns, the idiomatic workarounds for language limitations that have persisted for years because nobody wanted to break the ecosystem.

When an LLM generates a TypeScript class, it is not reasoning about types or control flow from first principles. It is pattern-matching against the accumulated output of human programmers who were themselves pattern-matching against a language designed for a machine. Three layers of indirection, none of them designed for clarity.

We gave the most capable language-processing systems in history a programming language that was designed, accidentally, layer by layer over sixty years, for compilers.

The Question

What would a language look like if you started from the actual requirement?

Not: "humans need to type this efficiently." Not: "the compiler needs to parse this in linear time."

Not: "this needs to feel familiar to C programmers."

The actual requirement, today, is this: an AI system needs to generate verifiable, correct code from a human specification, and that code needs to run on real hardware.

The generation step is the interesting one. When a human writes code, the language needs to be writable, because humans are slow, make typos, and need mnemonic help. When an AI generates code, none of those constraints apply. An AI does not get tired. It does not mistype. It does not need `for` to remind it that it is iterating. It needs a language where the structural properties of

correct code are computable, verifiable, and unambiguous, so that what it generates can be mechanically checked against what you asked for.

The language also needs to be reviewable. Humans are not leaving the loop. We need to look at AI-generated code and decide whether it is correct. That is not the same as writability. A language optimized for review is terser than one optimized for writing. It externalizes intent rather than process. It makes the "what" obvious even when the "how" is dense.

This is a real design constraint, not a philosophy. And no existing language was designed for it.

Loom

I have been calling the answer Loom.

The name comes from work I did earlier on Generative Specification, which proposed that AI-assisted development needs a formal specification discipline to prevent structural drift. In that paper, the loom metaphor described specification as the structure that shapes what the weaver produces. Loom is a continuation of that idea taken one layer deeper.

The design principles are five:

Intent-preserving. Loom captures what a system should do, not how memory is laid out. There is no manual memory model, no stack discipline baked into the syntax. Those are compiler concerns.

Effect-transparent. Every function that touches the outside world declares it in its type signature. A function that reads from a database says so. A function that calls the network says so. A function that reads the clock says so. Purity is the default. Effects are explicit, traceable, and compositional.

Contract-first. Preconditions and postconditions are part of the function signature, not comments that rot. The compiler knows them. The type checker enforces them where it can. The runtime checks them where it cannot.

Refinement-typed. Types are not just shapes. An `Email` is not a `String`. A

`PositiveInt` is not an `Int`. Invariants are encoded in the type, verified at compile time where possible, checked at the boundary where not.

No ceremony. No null. No exceptions. No undefined behavior. No implicit coercions.

`Option<T>` for absence. `Result<T, E>` for failure. Match expressions that the compiler refuses to compile if they are not exhaustive.

The compilation path: Loom source compiles to a typed intermediate representation, passes through a type checker, effect checker, and contract verifier, then hands off to LLVM. From LLVM, it reaches native binaries, WASM, GPU kernels. The human-legibility layer is removed from the execution path entirely, because that layer was never for the machine anyway.

Where Humans Belong

This is not about removing humans. The proposal is precisely the opposite.

The Generative Specification stack looks like this: humans write intent in natural language and structured specs. ForgeCraft validates that AI output satisfies those specs. Loom is what the AI generates when the target language is designed to make spec satisfaction mechanically verifiable.

In that stack, humans operate at the level of intent, contracts, and judgment. They decide what the system should do. They review whether the generated Loom matches what they asked for. They judge whether the contracts are complete. That is the right level. That is where human cognition adds value: at semantics, at goals, at the question "is this actually what we wanted?"

Humans are not good at catching off-by-one errors in import paths. Humans are not good at remembering which version of Node handles which edge case in the event loop. These are things we have spent sixty years training ourselves to do because the machines needed us to do them. They did not need us for our reasoning. They needed us as translators.

The Last Line

The Prussian school system trained factory workers. The 20th century knowledge economy trained information workers. The programming culture of the last forty years trained compiler messengers, people who learned to think like machines so they could communicate between human intent and machine execution.

The right future is not programmers who think faster. It is humans who think at the level of intent and judgment, and machines that handle the translation. That future needs a language designed for the translation layer, not accidentally inherited from a line of messengers going back to FORTRAN.

Loom is what execution looks like when we stop pretending humans and machines want the same thing from a programming language.

They never did.